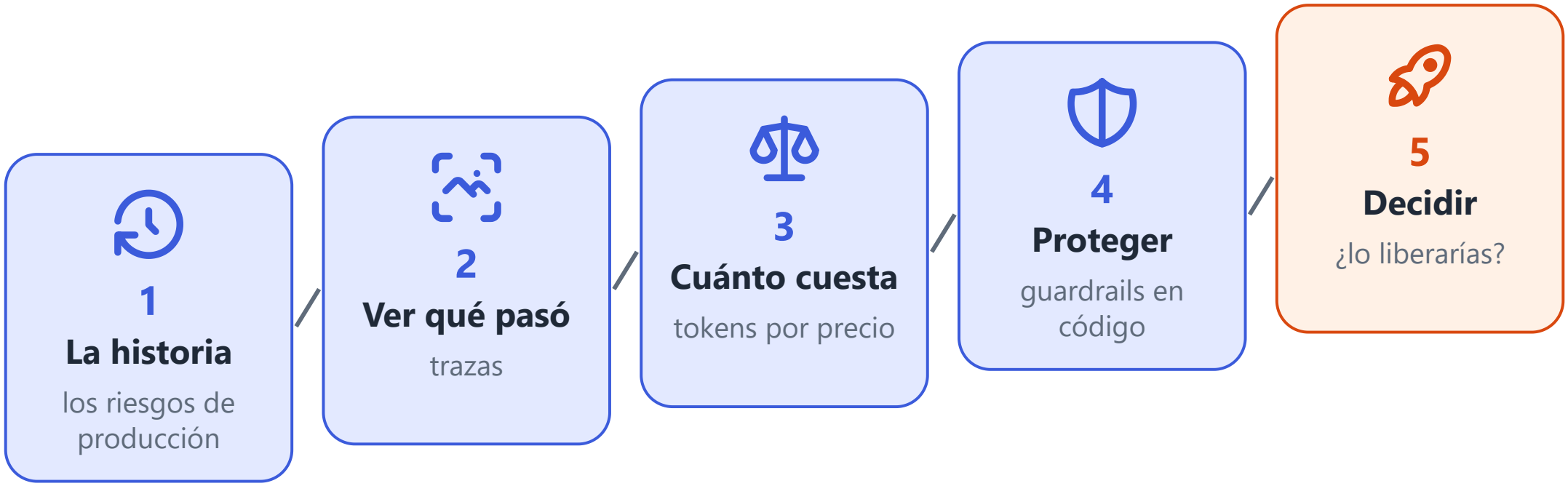
Producción

M8: v1.0, Producción



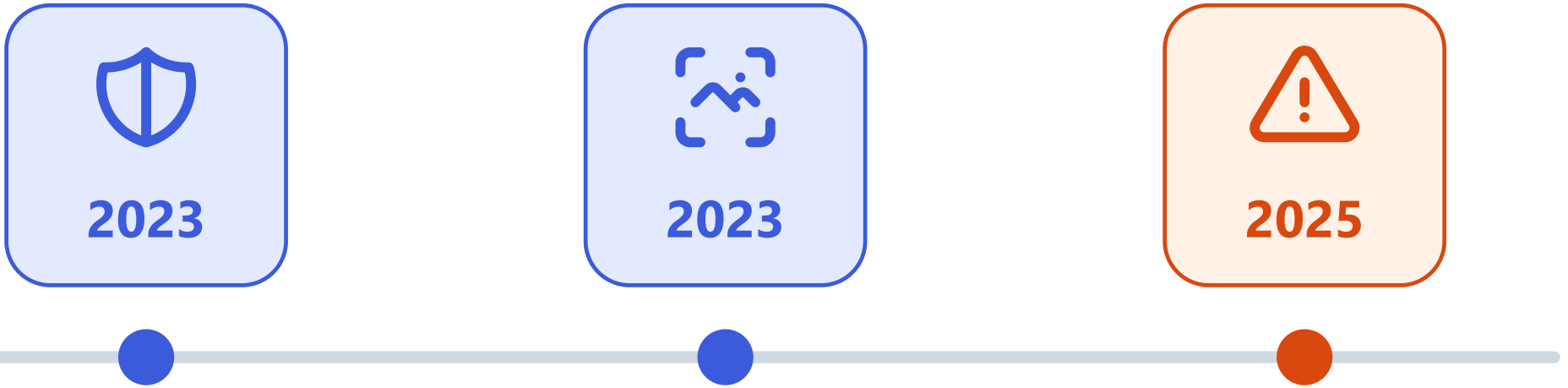
Hoy: el recorrido

De «anda en mi máquina» a «lo liberaría»



Venís de v0.5: un equipo con aprobación humana. Hoy juntás la evidencia para liberarlo.

Producción trajo sus propios riesgos



Es el texto 7 de la clase 2: una orden escondida en los datos.

La lista: OWASP Top 10 para LLM.

Sin mirar: ¿qué no podrías explicar?



Si falla mañana

con un usuario real, ¿qué parte no podrías explicar?

?

Anotala: es lo primero que hay que poder ver.

¿Qué parte no podrías explicar?

Respuestas: traza, costo y guardrail

Traza

qué herramienta usó y con qué datos

Tokens y precio

cuánto costó esa consulta

Guardrail

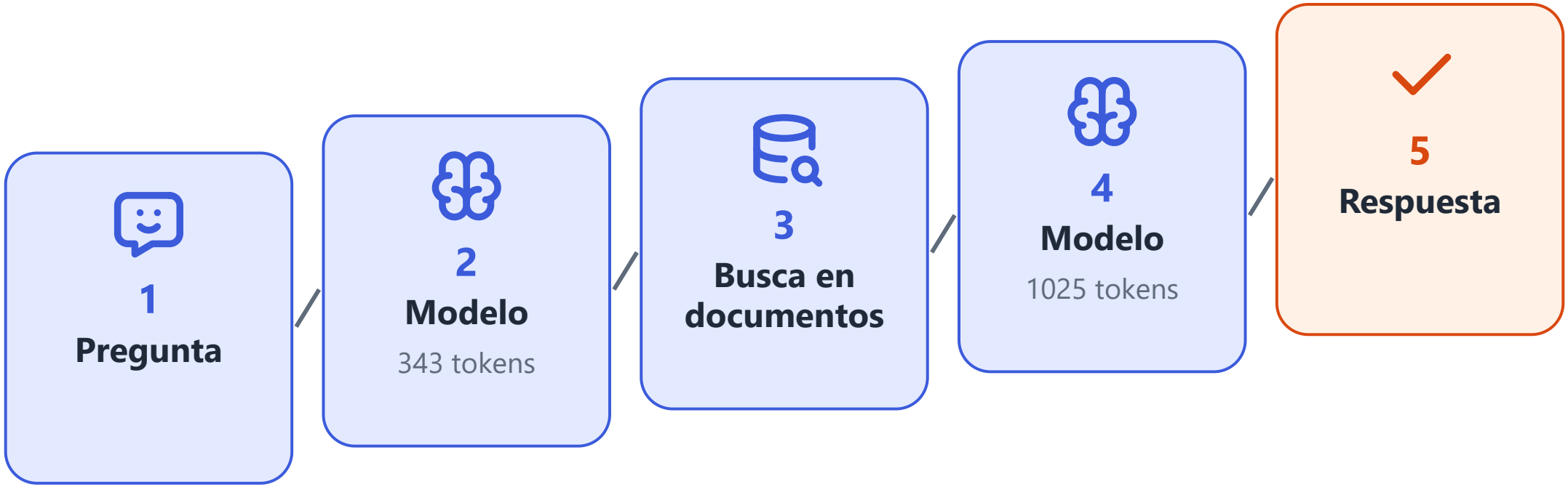
por qué dijo lo que no debía, con su test

Producción: el agente con sus controles



Lo usa otra persona: cada consulta deja traza y costo.

Una traza muestra cada paso de una consulta



Traza: el registro de cada paso, como la caja negra de un avión.

Cada paso queda como un renglón de `logs/traza.jsonl`.

Langfuse es una de varias herramientas de trazas

- **La** — Langfuse, local
- LangSmith (LangChain)
- **Ph** — Phoenix (Arize)
- Weave (W&B)
- **He** — Helicone

Todas guardan lo mismo: cada llamada, sus herramientas y sus tokens.

Cada renglón de la traza es un paso

```
{"tipo": "consulta", "hilo": "v10-dfa2144899943"}
{"tipo": "modelo", "uso": {"input_tokens": 326, "output_tokens": 17, …}}
{"tipo": "herramienta", "nombre": "buscar_documentos", "entrada": …'interrupt'…}
{"tipo": "modelo", "uso": {"input_tokens": 881, "output_tokens": 144, …}}

tokens: entrada=1207 salida=161 · costo US$ 0.000185 · por mes (1000/día): US$ 5.55
(precios de ejemplo: los reales son más altos)
```

- Gemini 3.5 Flash-Lite, v1.0.
- Con precios reales (19/10), unos US$ 23 por mes.

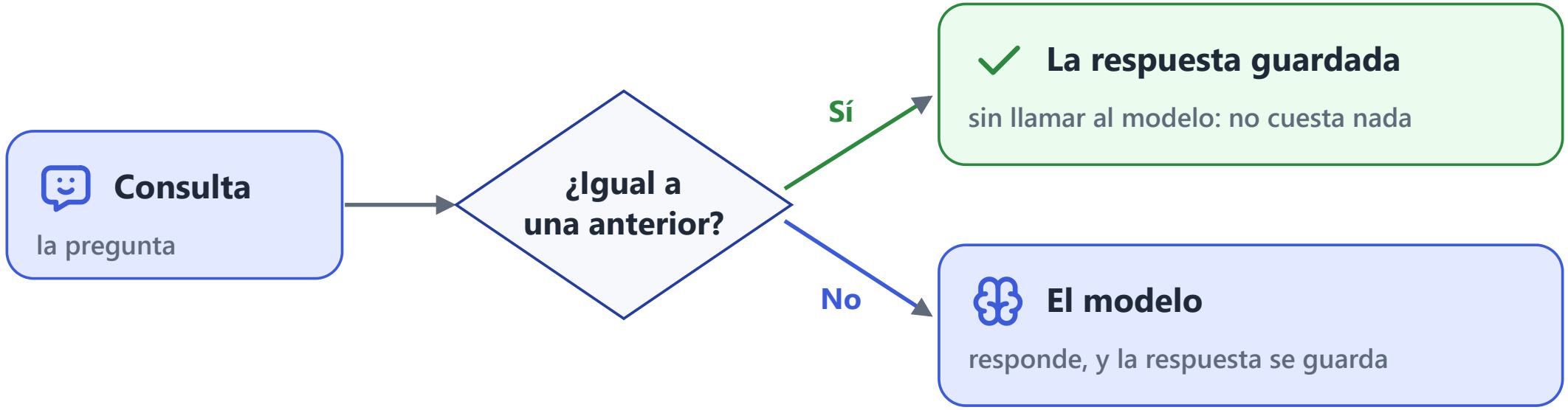
▶ `uv run python -m ejemplos.clase8 traza`

Del token al mes: multiplicá y proyectá

	Por consulta	1000 por día, 30 días
Tokens de entrada	1207	36 millones
Tokens de salida	161	4,8 millones
Costo*	US$ 0,000185	US$ 5,55

* Precios de ejemplo: US$ 0,10 y 0,40 por millón.

Una caché responde lo repetido sin el modelo



Riesgo: devolver algo viejo; y si compara por parecido, algo equivocado

Ahorra costo: lo repetido no llama al modelo.

Repetida: 0 segundos. Con una minúscula: al modelo

```
set_llm_cache(InMemoryCache())    # guarda por el texto exacto del pedido

1.31 s · tokens 52 · «¿Qué hace interrupt()…»
0.00 s · tokens 52 · «¿Qué hace interrupt()…»    ← la guardada
0.84 s · tokens 54 · «¿qué hace interrupt()…»    ← distinta: otra llamada
```

- gpt-5.4-nano. La guardada trae sus tokens viejos: si sumás el costo con esos números, contás de más.

▶ `uv run python -m ejemplos.clase8 cache`

Un guardrail en código cambia la respuesta

Sin guardrail
web con ERROR →
«es la 9.9»

→

Con guardrail
«No pude completar.
No invento el dato.»

- **Guardrail** = un control en código, como una baranda.
- Si inventa sin pedir la web, pasa: ¿qué test lo muestra?

Más guardrails que los de LangChain

Middleware de LangChain

NeMo Guardrails (NVIDIA)

Guardrails AI

Llama Guard (Meta)

Revisan lo que entra o sale. Ninguno ve lo que no previste.

El fallo seguro revisa al final

```
@after_agent
def fallo_seguro(estado, runtime):
    caidas = […startswith("ERROR")]
    if caidas and not any(… DECLARA_FALLO):
        "No pude completar… No invento el dato."
```

- Al final del agente
- Revisa la respuesta
- ¿Una herramienta dio ERROR?
- ¿Y la respuesta no lo dice?
- Se reemplaza

Si esconde un error, va un aviso honesto.

El email se tapa antes de llegar al modelo

Escribe la persona
«Mi mail es ana.perez@ejemplo.com»

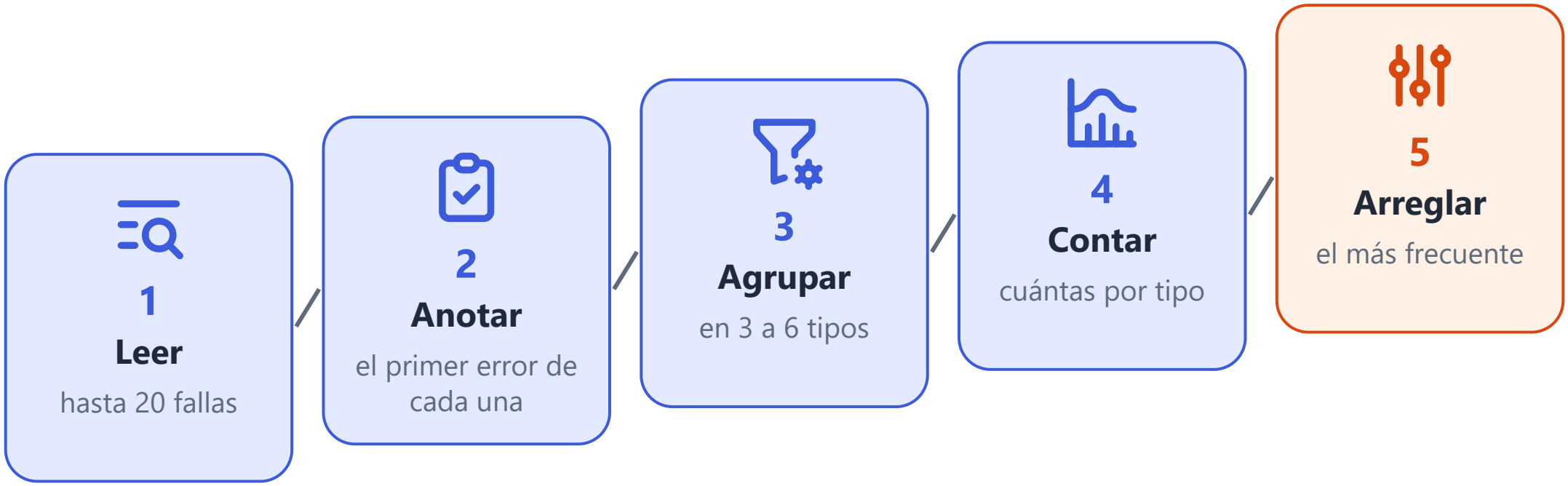
→

Ve el modelo
«Mi mail es [REDACTED_EMAIL]»

Sin consentimiento, no sale (Ley 18.331). ▶

```
uv run python -m ejemplos.clase8 email
```

Antes de medir más, leé las respuestas que fallaron



Como mirar los residuos en ML antes de cambiar el modelo.

Cada tipo de error pide su propio control



Sigue una regla

se controla con código



Pide criterio

un modelo juez: sí o no



Juez nuevo

comparalo con 20 casos anotados

Sí o no: un puntaje de 1 a 10 no dice qué arreglar.

Modelo chico en v0.5: las 9 fallas, al elegir

```
CSV anotado (4 de las 9 filas)
id  pregunta (recortada)                    usó → esperaba                  primer error
1   ¿Qué hace interrupt() en LangGraph?      nada → buscar_documentos        no buscó
3   ¿Qué clase borra mensajes del estado?    calcular → buscar_documentos    equivocada
5   ¿Última versión de LangGraph en PyPI?    calcular ×3 → buscar_web        equivocada
11  Gracias, eso es todo.                    buscar_documentos → nada        de más
2.ª corrida: la cuenta
primer_error
no buscó en los documentos           4
pidió una herramienta equivocada     4
usó una herramienta sin necesidad    1
```

- Modelo local chico (3 de 12).
- Ninguna fue «buscó bien y respondió mal».

▶ `uv run python -m ejemplos.clase8 errores datos/suite-v05-chico.csv`

Análisis de errores con tu suite, en dos corridas

```
uv run python -m ejemplos.clase8 errores logs/suite-v05.csv   # 1.ª vez: lista
uv run python -m ejemplos.clase8 errores logs/suite-v05.csv   # 2.ª vez: cuenta
```

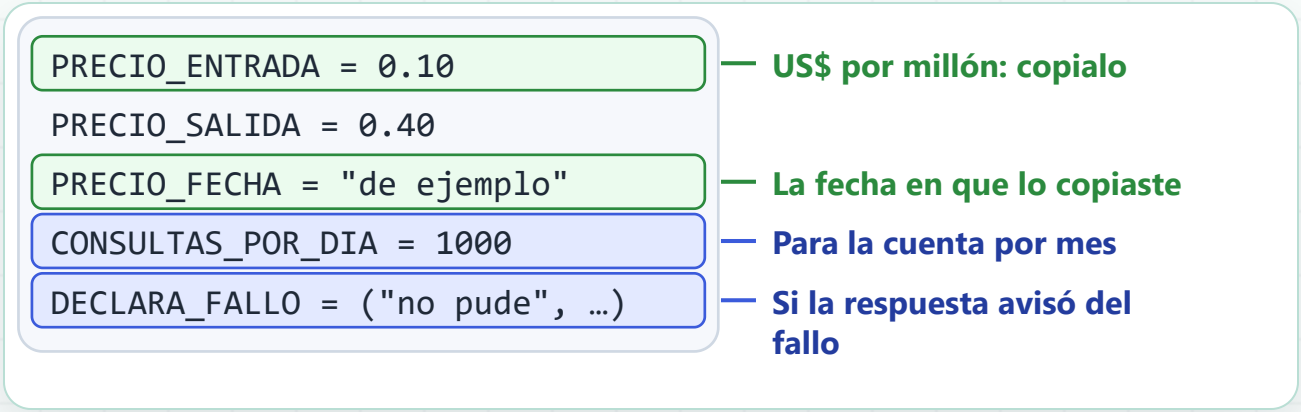
- Entre las dos, completá la columna `primer_error` de `logs/suite-v05-errores.csv`
- Arreglá primero el tipo más frecuente.

Taller v1.0: traza, costo y tu peor fallo (25 min)

1. **Suite** con v1.0, web grabada.
2. **Costo** con 3 preguntas.
3. **Test** de tu peor fallo.

Práctica M8, pasos 1 a 3.

Primer control · 10–15 min: traza con herramientas y tokens.



Repaso: hoy te llevás tres ideas

Traza

ver cada paso y sus tokens

Costo

tokens por precio, al mes

Guardrail

un control en código

Antes de medir más, leé las respuestas que fallaron.

¿Qué te impide liberarlo hoy?

Nombrá lo que hoy te frena y la prueba que lo resolvería.

- Anotá tu **confianza (1 a 4)**.
- Entregás **hasta el dom 22/11, 23:59**; la lista, en la tarea «Entrega final» del aula virtual.